

①

CSS411 Cryptography and Network Security

Assignment.

2023BCSO143

Sept-29-2026

Yash Prashant Sonawane

Batch-2

Q1 In the case of an E_c defined over a finite field of integer modulus 23 given by $y^2 = x^3 + 3x + 8$ Given $P = (0, 10)$ $Q = (7, 2)$, calculate $P + Q$

$$\Rightarrow y^2 = x^3 + 3x + 8 \pmod{23}$$

$$P = (0, 10) \quad Q = (7, 2)$$

$$\text{for } P \neq Q \quad \lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{23} = \frac{2 - 10}{7 - 0} = \frac{-8}{7}$$

modulus 23

$$= -8 \equiv 15 \pmod{23}$$

$$7^{-1} \equiv 10 \pmod{23}$$

$$\text{as } 7 \times 10 = 70 \equiv 1 \pmod{23}$$

$$\lambda = 15 \times 10 = 12 \pmod{23}$$

$$x_3 = \lambda^2 - x_1 - x_2$$

$$x_3 = 12^2 - 0 - 7 = 144 - 7 = 137 \pmod{23} = 22$$

$$x_3 = 22$$

$$y_3 = \lambda(x_1 - x_3) - y_1 = 12(0 - 22) - 10 = -264 - 10 = -274$$

$$-274 \pmod{23} = 2$$

$$P + Q = (22, 2)$$

(2)

Q2 Check whether the following point are on elliptical curve $y^2 = x^3 + 5x + 7$ over finite field modulo 23

(a) $(1, 6)$ Hint: check whether $y^2 \bmod 23 = (x^3 + 5x + 7) \bmod 23$
 (b) $(1, 5)$

(a) $P = (1, 6)$

$$\text{Left side: } y^2 = 6^2 = 36$$

$$36 \bmod 23 = 13$$

$$\text{Right side: } x^3 + 5x + 7$$

$$= 1^3 + 5(1) + 7 = 13$$

$$\text{LHS} = \text{RHS} = 13$$

$\therefore (1, 6)$ lies on the elliptical curve.

(b) $P = (1, 5)$

$$\text{Left side: } y^2 = 5^2 = 25$$

$$25 \bmod 23 = 2$$

$$\text{Right side} = 1^3 + 5(1) + 7 = 13$$

$$2 \neq 13$$

$\therefore (1, 5)$ does not lie on the elliptical curve.

(3)

Q3 In the case of an EC define over a finite field of integer modulus 17 given by $y^2 = x^3 + 2x + 3$ Given $P = (5, 11)$ calculate $2P$ & $3P$

$$\Rightarrow y^2 = x^3 + 2x + 3 \pmod{17}$$

$$P = (5, 11)$$

For point doubling

$$\lambda = \frac{3x_1^2 + a}{2y_1} \quad a=2 \quad x_1=5 \quad y_1=11$$

S1 = Calculate slope

$$\lambda = \frac{3(5)^2 + 2}{2 \times 11} = \frac{77}{22} = \frac{77}{22}$$

$$\text{modulus } 17: \quad 77 \bmod 17 = 9$$

$$22 \bmod 17 = 5$$

$$\lambda = \frac{9}{5}$$

Inverse of 5 modulo 17

$$5^{-1} = 7$$

$$\text{because } 5 \times 7 = 35 \equiv 1 \pmod{17}$$

$$\text{Therefore } \lambda = 9 \times 7 = 63 \equiv 12$$

Step 2 calculate x_2

$$x_2 = \lambda^2 - 2x_1 = 12^2 - 2(5) = 144 - 10 = 134$$

$$134 \bmod 17 = 15$$

$$\boxed{x_2 = 15}$$

(4)

Calculate y_2

$$\begin{aligned}
 y_2 &= \lambda(x_1 - x_2) - y_1 \\
 &= 12(5 - 15) - 11 \\
 &= -120 - 11 = -131 \\
 &= -131 \bmod 17 = 5
 \end{aligned}$$

$$\therefore 2P = (15, 5)$$

Now $3P \div$

$$3P = 2P + P$$

$$2P = (15, 5) \quad P = (5, 11)$$

$$\text{Slope } \lambda = \frac{11 - 5}{5 - 15} = \frac{6}{-10}$$

$$\text{modulo } 17: \quad \cancel{6} \div 7$$

$$\text{because } -10 \equiv 7 \pmod{17}$$

Inverse of 7 modulo 17 is 5

$$7 \times 5 = 35 \equiv 1$$

$$\lambda = 6 \times 5 = 30 \equiv 13$$

$$\text{Then } x_3 = 13^2 - 15 - 5 = 169 - 20 = 149$$

$$149 \bmod 17 = 13$$

$$\text{and } y_3 = 13(15 - 13) - 5$$

$$= 26 - 5 = 21$$

$$21 \bmod 17 = 4$$

Therefore:

$$3P = (13, 4)$$

Q 4

(5)

(a) Hash of "Hello"

$$H=72, E=69, L=76, L=76, O=79$$

$$72 + 69 + 76 + 76 + 79 = 372$$

$$H("HELLO") = 372 \text{ mod } 100$$

$$= 72$$

(b) Modified message $M' = \text{"HeLmo"}$

(c) New hash value

Ascii Value

$$H=72, E=69, L=76, M=77, O=79$$

$$\text{Sum} = 72 + 69 + 76 + 77 + 79 = 373$$

$$373 \text{ mod } 100 = 73$$

(d) Can receiver detect modification

Original hash: 72

Modified hash: 73

Since: $72 \neq 73$

the receiver can detect this particular modification

Yes, the modification can be detected

e) Limitation

The toy hash function is not cryptographically secure because

it only uses the sum of ascii value modulo 100,

Different messages can easily produce the same hash (a collision)

So it cannot reliably protect message integrity against intentional attack.

Question 5.

Write a program using suitable cryptographic library functions to implement digital signatures using both Elliptic Curve Digital Signature Algorithm (ECDSA) and RSA Digital Signature.

- a) Generate an ECDSA public–private key pair using a standard elliptic curve such as SECP256R1.
- b) Generate an RSA public–private key pair with a key size of at least 2048 bits.
- c) Accept a message from the user.
- d) Generate a digital signature for the message using:
 - a. ECDSA with SHA-256
 - b. RSA with SHA-256 and an appropriate signature padding scheme such as RSA-PSS
- e) Display the generated signatures.
- f) Verify both signatures using the corresponding public keys.
- g) Modify the original message and demonstrate that signature verification fails for the modified message. Use suitable library functions available in Python's cryptography library or an equivalent cryptographic library.

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec, rsa, padding
from cryptography.hazmat.primitives.serialization import (
    Encoding, PrivateFormat, NoEncryption
)

message = input("Enter message: ").encode()

# ECDSA Key Generation
ecdsa_private_key = ec.generate_private_key(ec.SECP256R1())
ecdsa_public_key = ecdsa_private_key.public_key()

# RSA Key Generation
rsa_private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
rsa_public_key = rsa_private_key.public_key()

# ECDSA Signature
ecdsa_signature = ecdsa_private_key.sign(
    message,
    ec.ECDSA(hashes.SHA256())
)
```

```

)

# RSA Signature using RSA-PSS
rsa_signature = rsa_private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("\nECDSA Signature:")
print(ecdsa_signature.hex())

print("\nRSA Signature:")
print(rsa_signature.hex())

# Verify ECDSA signature
try:
    ecdsa_public_key.verify(
        ecdsa_signature,
        message,
        ec.ECDSA(hashes.SHA256())
    )
    print("\nECDSA Verification: Successful")
except Exception:
    print("\nECDSA Verification: Failed")

# Verify RSA signature
try:
    rsa_public_key.verify(
        rsa_signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    print("RSA Verification: Successful")
except Exception:

```

```

    print("RSA Verification: Failed")

# Modified message
modified_message = input("\nEnter modified message: ").encode()

# Verify ECDSA with modified message
try:
    ecdsa_public_key.verify(
        ecdsa_signature,
        modified_message,
        ec.ECDSA(hashes.SHA256())
    )
    print("Modified ECDSA Verification: Successful")
except Exception:
    print("Modified ECDSA Verification: Failed")

# Verify RSA with modified message
try:
    rsa_public_key.verify(
        rsa_signature,
        modified_message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    print("Modified RSA Verification: Successful")
except Exception:
    print("Modified RSA Verification: Failed")

```

```

● yashsonawane@yashs-MacBook-Air cryptography % python3 que1.py
○ yashsonawane@yashs-MacBook-Air cryptography % python3 que1.py
Enter message: hello Crypto

ECDSA Signature:
3045022100ea1921b188bd0d528aedd6f7410e9952b9a7556d36ebacf51fd05b8661f7586c022078dac1d6481742d3986e79c74a610ad17b560d80426b2dd6cd5fd186121e6139

RSA Signature:
8cc4c6ac98932243a11fcf463133021ceeb3a256ed004316b3b5dbc012724dec30970c1b421b328832d7ca5d0ef662a8c79c338af0c178df65981a5a72679848c2692a03fe8f6224e82b64446f97
45ebd9a18c7f71c02440dfbcdbe21794c09435c3de5b1e0688c46adeec0cda00f63c5556d6d883787185ee11f3742586499f7813d68ca9bae844eb25173ab491f35d1b4e83591d03e0b28ef407ba
ac495e31db8b9b287e4cfb5b1cf5cd6712abcdaa37f3935dc48350212faa270267faa33015262459c36ab3ae0ecae0e57c7bad0ba7983a7cc43bbe2e08e468eac14bf6b9941cfc7563822a635acf
19b7bac64d7d1434e05f05ebaa5b08b2f89e882df9a3

ECDSA Verification: Successful
RSA Verification: Successful

Enter modified message: 

```